Constructing a Hybrid Software Process Simulation Model in Practice: An Exemplar from Industry

Yue Li, He Zhang, Liming Dong, Bohan Liu and Jinyu Ma
yueli.dom@gmail.com,hezhang@nju.edu.cn,lmdongmg@gmail.com
dg1732003@smail.nju.edu.cn,jinyu-ma@foxmail.com
State Key Laboratory of Novel Software Technology, Software Institute, Nanjing University
Nanjing, China

ABSTRACT

Background: Software Process Simulation Modeling (SPSM) is of paramount importance to support quantitative management of software development process. Hybrid process simulation combines multiple simulation paradigms to reflect complex changes in realistic software processes, which brings inherent challenges to process management. Constructing a hybrid model requires more modeling expertise and experience than modeling by solo-paradigm. However, a few studies explicitly discuss the challenges they encountered as a topic, which may discourage practitioners. *Objective:* Our aim in this study is to present an industrial process modeling project as an exemplar to demonstrate and discuss the technical issues and challenges associated with hybrid process simulation in practice. *Method:* Based on the collaboration with a global software enterprise, we constructed a hybrid process simulation model that combines System Dynamics (SD) and Discrete Event Simulation (DES) to predict the project duration and release date for management. *Results:* Several challenges around hybrid process simulation of software development process are identified and discussed with the proposal of sets of solutions from different perspectives. The model is validated by comparing the simulation result with the actual enactment of the process in industry. In addition, the result confirms the rationality and efficacy of the suggested solutions to some extent. *Conclusions:* In the collaboration with the enterprise, five-step modeling procedure was adopted for constructing the hybrid process model. The experience reported about the detailed steps of hybrid modeling may offer reference value to the SPSM community.

KEYWORDS

software process modeling, hybrid simulation

ACM Reference Format:
Yue Li, He Zhang, Liming Dong, Bohan Liu and Jinyu Ma. 2020. Constructing a Hybrid Software Process Simulation Model in Practice: An Exemplar from Industry. In *International Conference on Software and Systems Process (ICSSP '20), October 10–11, 2020, Seoul, Republic of Korea.* ACM, Seoul, Korea, 10 pages. https://doi.org/10.1145/3379177.3388906

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
ICSSP '20, October 10–11, 2020, Seoul, Republic of Korea
© 2020 Association for Computing Machinery.
ACM ISBN 978-1-4503-7512-2/20/10…$15.00
https://doi.org/10.1145/3379177.3388906

1 INTRODUCTION

Software Process Simulation Modeling (SPSM) was introduced to the community by Abdel-Hamid et al. [1] and other pioneers in 1980s as a supporting tool for quantitative software process management. Zhang et al. [35] traces the evolution in SPSM research from 1998 to 2008 and found that the three main paradigms in SPSM community are System Dynamics (SD), Discrete Event Simulation (DES) and Agent-Based Simulation (ABS). These paradigms differ in their worldviews and levels of abstraction [21], and each of them comes with their shortcomings. For example, SD could clearly represent the relationships between dynamic variables. However, it is usually limited to simulate processes at high abstraction, because distinguishing different modules will multiply the model complexity [16], [21]. A DES model, which simulates a system as networks of queues and activities and treats each type of the modules as a unique entity, specializes in simulating the specific entity flows of the system. However, it is not able to continuously maintain dynamic variables at the overall system scope, such as productivity, detection rates, etc. [13]. Specifically, ABS is able to capture the characteristics of the interactions between the individuals and their effects, while it is hard to represent the effects of feedback loops and the relationships among dynamic variables.

Hybrid process simulation models have attracted interests as it can accommodate complex processes and data by taking into account the benefits of different modeling paradigms. Gao et al. [13] conducted a systematic literature review on hybrid software process simulation models, and identified four typical combinations of simulation paradigms, in which the combination of SD and DES has become the most frequently used hybrid simulation approach in the SPS community. Since hybrid model was first introduced by Farhland in 1970 [11], hybrid simulation has been applied in different research domains, such as manufacturing [26], supply chain management [34], healthcare [6] and software industry [8] etc. In Software Engineering (SE) many studies have tried to apply the hybrid modeling by SD+DES to model software process more realistically [8], [12], [4], [23]. The hybrid simulation provides a better insight into the system in hand as it allows modelers to assess its inherent challenges from different dimensions [37], [10]. Silva Filho et al. [12] presented the development of an approach to facilitate software process simulation modeling in small and mid-sized enterprises and describes the experience aimed at identifying weaknesses and improvement opportunities to support the approach evolution.

Despite the continuous effort in hybrid simulation research and practice, it is found that many modelers do not report simulation modeling process in literature [2]. Previous studies [3], [24] also

emphasize that having a detailed description of simulation modeling process with identified activities and artifacts could be the basis to identify how simulation models could be constructed for and with reuse, which could reduce effort and cost of adopting SPSM. Besides, existing attempts in hybrid simulation practice are ad hoc and pragmatic with no clear guidelines or methodology [37]. In this paper, we propose a hybrid modeling approach by constructing an SD+DES model and introduce the detailed simulation modeling process and evaluate its use in an industry exemplar for exploring the difficulties of hybrid simulation modeling in industry settings. By reporting the exemplar, we offer our practical experiences for process modelers in practice.

The rest of this paper consists of: Section 2 presents the related work of hybrid simulation on software process domain. Section 3 introduces the example case in the study including the development process and data source. In Section 4, we describe the whole process from demand identification, model construction to verification and validation. Section 5 discusses the experience in constructing hybrid model in industry context. Section 6 draws the conclusion.

2 RELATED WORK

Hybrid simulation has been applied in many studies on software process. Also several studies have proposed hybrid simulation frameworks to guide the development of hybrid process model. This section introduces the most relevant work in the area of hybrid simulation for modeling software process.

Martin and Raffo [22] developed a hybrid model of software development process and simulated the effects of resource constraints on parallel activities. They presented a hybrid model to evaluate simultaneous changes to both the development process and the project environment. The risks coming with process changes were mitigated by the evaluation of potential process changes using simulation.

Lakey et al. [19] developed a hybrid model that consists of SD and DES. The hybrid model was constructed for software project estimation and management in a specific software organization. However, their hybrid model was not verified in practice.

Choi et al. [8] proposed their hybrid modeling approach and constructed a Waterfall like hybrid software process simulation model as an example for validating the approach. Their solution is an enhancement to Lakey et al.'s [19] model with the refined time advance. By constraining the time advance of discrete-event model within a small enough range of time, the approach is able to model differential equation systems with discrete-event modeling.

Nauman et al. [3] described a consolidated process for SPSM and presented the steps of the simulation modeling and provided guidelines for each step through a review of literature. Based on the experience with using these guidelines in a large Telecommunication vendor, they complemented the consolidated modeling process.

Raffo et al. [28] discussed some of the important empirical issues that arise in SPSM. Based on their practical modeling experiences, various statistical and analytical techniques explored could be applied to evaluate process alternatives.

The existing research mainly discussed simulation paradigms for modeling software processes or software process improvement by using simulation models. Different from these studies, our research focuses on the process of constructing a hybrid model in practice by illustrating an industrial exemplar as well as reports the challenges in the hybrid modeling and our experiences.

3 THE EXEMPLAR CASE

This study was conducted in the collaboration project with a global ICT enterprise that operates in more than 170 countries and regions. Its business covers multiple domains of ICT, which implement a variety of software development processes. In order to construct a relatively universal software process model, we selected one of their most typical software development processes, which is an incremental development process.

One of the outcome of the collaboration project is a process simulation model for project schedule planning and management. This section describes the software process modeled and the available data we investigated by analyzing the data from repositories and a series of interviews with the practitioners.

3.1 Project selection

The enterprise is composed of various divisions responsible for different lines of business distributed throughout the world. We selected one of their software divisions for the easy access for regular communication and investigation. Based on the analysis of the data quality and accessibility, we selected one mobile application development project as the pilot modeling study. The development process and repositories they used are described in the rest of this section.

3.2 Development process to be modeled

Basically there are three main phases in each increment of the development process to be modeled: *design*, *implementation*, and *test* phases.

Design phase. At the beginning of the development, the Project Manager (PM) organizes a joint kick-off meeting with designers, developers, and testers to determine the System Requirements (SRs). Then, software designers decompose SRs into fine-grained *stories* with comparable workloads. *Story* is the smallest unit of software requirements to be developed in the enterprise as well as to be modeled as the input to the process simulation model.

Implementation phase. One particular *story* is assigned to specific developer(s), who is responsible for requirement learning, coding, self-testing, and then committing the code for multiple rounds of testing.

Test phase. Once the *stories* are assigned to the test team, they start to create test cases for testing these *stories* accordingly. When a code submission arrives, the testers should conduct Story Test (ST) and generate defect reports if any defects are detected. Then, the relevant developers would receive the particular defect reports. After all the defects from ST are fixed, the tester should conduct System Integration Test (SIT). Once the SIT is passed, the corresponding requirements that is composed of the passed *stories* can be released.

3.3 Data sources for simulation

Process data is fundamental in building a process simulation model. The available data that might be used in simulation modeling are from three different sources: online software repositories, offline documents, and inquiries of project team members, e.g., developers, testers, PM and Test Project Manager (TPM).

Online software repositories. The online software repositories in the enterprise are primarily used to manage software artifacts including requirements, defects, and test cases. Some data in the repositories might be used for process modeling but with many challenges in reality. There are four main repositories that can contribute to modeling the software process. *Requirement Management System* (RMS) records and tracks the artifacts generated in the life cycle of requirements including requirement review, requirement scheduling, requirement decomposition, design, development, test and so on. *SRs* are decomposed by designers into one or more *stories*, the smallest unit of requirements recorded in RMS. The process data associated with requirements in RMS contains information such as the transfer time of *story* and the roles involved in different activities. *Defect Tracking Systems* (DTS) records the defects detected in test phase. It stores the meta-data of a defect with all the involved staff. *Test Case Management System* (TCMS) helps testers manage and maintain the test cases used in the software process. *Test Process Management System* (TPMS) is a Kanban system that manages and visualizes the test process, records and tracks the status of the test and the follow-up schedules, including 'toDo', 'test plan', 'execute test case', and 'finish'.

Offline software documents. Apart from the online structured data gathered from the repositories, documentations can provide supplementary information for process modeling. Besides online tracking of requirements, PM maintains the *Requirement Tracking DOCument* (RTDOC) to track the specified activities of each requirement during requirement development process. After the planning meeting each *story* is assigned to developers. Each developer records the start-time and end-time of requirement learning, end-time of coding and self-test, and actual transfer test time on RTDOC. *Test Planned Schedule DOCument* (TPSDOC) is designed at the beginning of the requirement development (RD). Test process data in TPSDOC contains information like the planned execution sequence, planned start-date and end-date of story test (ST).

Inquires to project team members. In some cases, the desired data for process modeling is not available in repositories and documents, the experience of project members would be a necessary complement. To understand the development process in the enterprise, we interviewed the PM, developers, TPM and testers to investigate the development activities they were involved and the usage of the online systems such as RMS, DTS, TCMS and TPMS. We selected RTDOC, TPMS and DTS as the main data sources in this study for the consideration on the data quality, e.g., the consistency between recorded data and enacted process. For example, developers might not transfer the requirement on RMS by themselves or update the requirement status on time. We further chose the RTDOC and TPMS as the supplements of RMS since they record the finest-grained requirements and help collect the requirements process variables during the entire development process. Moreover, RTDOC and TPMS record more realistic data as each phase was managed and controlled by PM or TPM respectively.

4 CONSTRUCTION OF A HYBRID SIMULATION MODEL IN PRACTICE

In this section, we describe our modeling process, which lasts nearly half year in collaboration with the enterprise. We also share the practical modeling issues in the process and the valuable experiences gained.

With the reference to the modeling steps discussed by Ali et al. [3], we elaborate our modeling process with five steps to build the hybrid model (as shown in Table 1) within the context of the project. Apart from modelers, we regard that the participants in modeling included the leaderships of the development projects and the engineers in the project teams.

Table 1: Steps of process modeling for hybrid simulation

Modeling Step in Practice	Participants per Step
A.Identifying Organization's Demand	Leaderships, Modelers
B. Designing Process Model	Leaderships, Modelers
1) Determining Model Granularity	Leaderships, Modelers
2) Determining Model Scope	Leaderships, Modelers
3) Choosing Simulation Paradigms	Modelers
4) Defining Model Structure	Modelers
C. Quantifying Model's Variables	Modelers, Engineers
1) Specifying Variables	Modelers
2) Calibrating Model	Modelers
D. Verification and Validation	Modelers, Engineers
E. Simulation Based Investigations	Modelers

4.1 Identifying organization's demand

As discussed in [3], [30], understanding the requirements of process management so as to identify the modeling purpose is the first step of process modeling. Our modeling purpose is to predict the release time of an increment of the simulated project. In the past, the common practice during the planning phase in the enterprise is to estimate the release time based on PM's personal experience without the support of any simulation tool or analytic model. Due to the poor accuracy of the manual estimation, it is common for development teams to work overtime towards the (estimated) agreed release dates. Furthermore, during the development, the re-planning needs to be made whenever any requirement changes and staff's turnover takes place. Therefore, both the organization and the development teams have urgent needs for quantitative management tools that can effectively support their (re-)planning.

4.2 Designing process model

4.2.1 Determining model granularity. Although Ali et al. did not discuss modeling granularity in their proposed steps [3], the decisions on modeling granularity were discussed in some other studies [18, 25, 29]. Since our modeling purpose is to predict the progress of releasing requirements, the modeling granularity needs to be

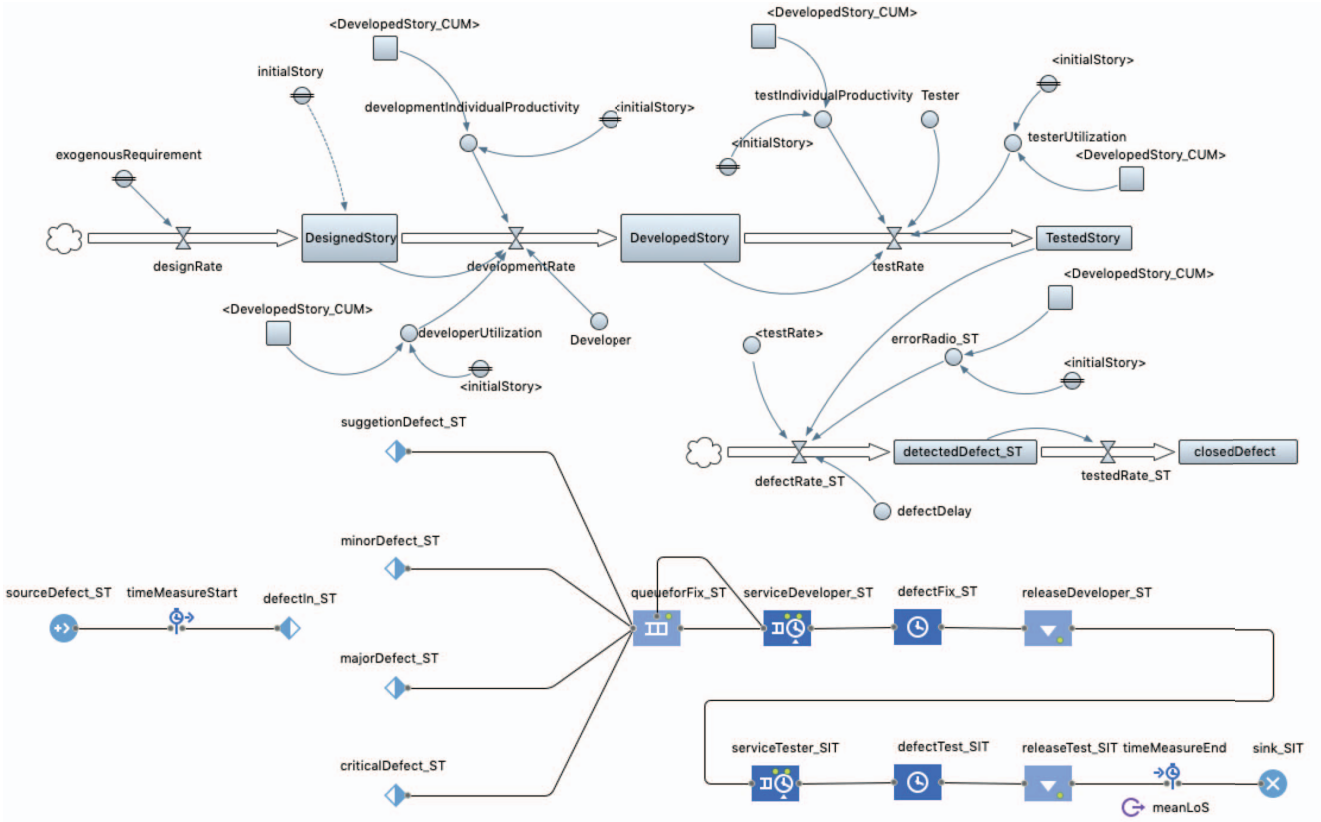


Figure 1: A hybrid software process simulation model

at an appropriate level of abstraction in corresponding to the description of requirements. As explained in Sec. 3.2, there are two different granular descriptions of requirements that recorded in the enterprise, i.e. *SR* and *story*. *SR* is the original requirements allocated to the specific development team, which can be split by the designer in the team into *stories* that can be implemented independently by a single developer in approximately two days. Since the split of *SRs* is solely based on the designer's personal experience without any criteria recorded, we were not able to accurately simulate the decomposition of *SRs*. Hence, the modeling granularity of requirements is at the *story* level in this study, that is, both the input (to be developed) and output (delivered) of the process model are *stories*. It should be noted that we did not chose Lines Of Code (LOC) to represent the size of software as many related studies (e.g., studies [17] and [9]) since the code size for requirements is not recorded in the enterprise's repositories.

4.2.2 Determining model scope. Although the model scope has to be large enough to address the proposed problem [3], [17], the modeler needs to invest extra effort to simulate an unnecessary portion of the software process when the model scope is oversized.

The main role of the simulation model is to assist PM in predicting the development progress over time. From this point of view, the model needs to cover the entire process (an increment) from the reception until the completion of the requirements. Hence, we decided that the modeling scope should contain design, implementation, and test phase. Since we were not able to simulate the split of *SRs*, only the design activity of *stories* is considered in the *design* phase.

4.2.3 Choosing simulation paradigms. As discussed in the [13], the three commonly used paradigms in SPSM are System Dynamics (SD), Discrete-Event Simulation (DES) and Agent-Based Simulation (ABS), of which SD is more suitable for simulating high abstraction macro-level processes as it can explicitly describe the continuous interactions between project factors. Given that the consistent and complete data is not available for some activities (events) of each phase of the development process, we chose SD for the model skeleton that is able to cover the entire increment of the *stories*.

In the specific development phases, we found that more structured data can be obtained for *test* phase than others as DTS records the life-cycle of defects and TCMS records the version control logs of test cases. In addition, the interviews with the developers and the testers indicate that the priorities and effectiveness differ among the four types of defects. Based on the statistical analysis, we found that these defect types also differ in numbers and the fixing effort consumed as shown in Table 2. The way developers fix them is different as explained in Section 3.2. In the enterprise, the priority of a defect is defined strictly according to its severity. High severity of defects (*critical* defects) are fixed by developers in time and as

early as possible. However, for the defects with lower severity (i.e. *major*, *minor*, and *suggestive* defects), developers do not fix them in a timely manner, so that the dwell time of the lower severity defects in the system looks longer than the higher in repositories. In order to capture more realistic characteristics of the process, we employed DES instead of SD to simulate the defect flows and the test-fixing activities of the defects, which can be regarded as discrete events at a fine-grained level.

As a result, we made the decision to construct a hybrid process simulation model that integrates SD and DES paradigms to achieve the modeling objectives in the condition of the limited available data unevenly distributed across the phases and the pursuit of high fidelity.

Table 2: Fixing time for different defect types of the project

Defect type	# Defects	Days in total	Days on average
Critical	2	0.4710	0.2355
Major	6	14.1540	2.3590
Minor	142	417.6770	2.9413
Suggestive	167	435.4440	2.6074

4.2.4 Defining model structure. As a continuous modeling technique, SD has been applied to study the interactions between software systems and environment [27]. The internal and external factors of the system are the main focus of SD paradigm [36]. Modelers need to analyze the factors and the interactions in between that may affect the process through data analysis, interviews, and other methods prior to model construction. DES has been adopted to research changes of event's state but occurring at random points, i.e. discrete times [13].

The total number of engineers participating in the development and their individual productivity are the primary factors affecting development rate of *stories* in SD. In addition, we observed that schedule pressure is also an influential factor for requirements development from an interview with developers.

Different from SD modeling, the parameters in a DES model that need to be considered are events and resources (entities). Hence, the time taken for defect fixing and regression testing is considered in modeling the *test* phase using DES. The major resources to be modeled in DES are the developers and testers who are involved in the *test* phase.

Figure 1 depicts the hybrid process model constructed for the project using the simulation tool Anylogic (ver.8.5) [1]. We applied the horizontal integration strategy [13] to integrate SD and DES paradigms in the hybrid model. In the figure, the upper part of the model depicts the SD module that simulates the *implementation* phase as well as the detection and the closing activities of defects; while the lower part shows the DES module that simulates the defect life cycle between detection and closing, including test and fixing activities. To be specific as follows:

[1]https://www.anylogic.com/

Implementation phase. Stories are handed over to developers when their design is completed. The main factors that determine *development rate* are *individual productivity of development* and *number of developers in the team.*

Test phase. Stories that have been implemented are passed to testers for testing. Similar to the *implementation* phase, *test rate* is determined by *individual productivity of the test* and *number of testers.* In addition, defect fixing and regression testing are included in the *test* phase. The detected defects are returned to developers for fixing, and the fixed defects are re-submitted to testers for regression testing.

4.3 Quantifying model's variables

After the definition of model structure, we next quantified the variables of the simulation model for execution with the data available in the enterprise.

Table 3 lists the raw data obtained directly from the different data sources. By analyzing the raw data, we could specify the values of the model's variables.

4.3.1 Specifying variables. For *implementation* phase, there are four main variables:

Number of developers records the total number of developers in an increment (release).

Developed stories denotes the total number of stories that have been developed in an increment.

Developer utilization means the ratio of the number of developers who participate in developing stories per day (N_{dev}) to the total number of developers (N_{t_dev}) in the team as defined in Equation 1. N_{dev} is measured by the number of developers who appear to be recorded in the online repositories or the offline documents for one day.

$$U_{dev} = \frac{N_{dev}}{N_{t_dev}} \tag{1}$$

Individual development productivity measures the number of stories developed per developer per day, which varies continuously as Equation 2, where N_{ds} is the number of stories developed per day.

$$P_{dev} = \frac{N_{ds}}{N_{dev}} \tag{2}$$

As for the *test* phase, the module includes eight major variables, among which the measurement of ***#testers***, ***#tested stories***, ***tester utilization*** and ***individual test productivity*** are similar to those in the *implementation* phase. The other four variables are enumerated follows:

#Detected defects is the number of detected defects after testing the stories.

Defect fix stay measures the live time of each type of defect from the detection by tester to the fixing by developers as calculated by Equation 3.

$$S_{bf} = \frac{\sum_{i \subseteq M}(D_{fi} - D_{ci})}{Size(M)} \tag{3}$$

where D_{fi} is the date that the developer fix the defect i, D_{ci} is the date that the tester detect the defect i, and M is the set of defects.

Table 3: Raw data for the hybrid process simulation model

Raw data	Description	Artifact	Role	Activity	Unit	Data source(s)
D_{sd}	Start date of the requirement development.	Req	Developer	Implemt.	Date	RTDOC/RTMS
D_{ed}	End date of the requirement development.	Req	Developer	Implemt.	Date	RTDOC/RTMS
EN_{dev}	Employee number of the responsible developer of the requirement.	Req.	Developer	Implemt.	Numeric	RTDOC/RTMS
D_{st}	Start date of the requirement test.	Req.	Tester	Test	Date	TPSDOC/TPMS
D_{et}	End date of the requirement test.	Req.	Tester	Test	Date	TPSDOC/TPMS
EN_{test}	Employee number of the responsible tester of the requirement.	Req.	Tester	Implemt.	Numeric	RTDOC/TPMS /TPSDOC
S_d	Severity of the defect.	Defect	Tester	Defect detection	Enum-type	DTS
T_{dd}	Detection time of the defect.	Defect	Tester	Defect detection	Time	DTS
EN_{ds}	Employee number of the defect submitter.	Defect	Tester	Defect detection	Numeric	DTS
T_{df}	Fix time of defect.	Defect	Developer	Defect fixing	Time	DTS
EN_{df}	Employee number of the responsible defect fixer.	Defect	Developer	Defect fixing	Numeric	DTS
T_{dr}	Regression time of defect.	Defect	Tester	Reg. test	Time	DTS
EN_{dr}	Employee number of the responsible tester for executing the regression test on the defect.	Defect	Tester	Reg. test	Time	DTS

Defect test stay measures the live time of each type of defect from fixing by developers until the testing by testers as Equation 4.

$$S_{bt} = \frac{\sum_{i \subseteq M}(D_{ti} - D_{fi})}{Size(M)} \tag{4}$$

where D_{ti} is the date that the tester tests the fixed defect i.
Defect detection ratio denotes the ratio of the detected defects to the total tested stories as Equation 5.

$$R_d = \frac{N_d}{N_{ts}} \tag{5}$$

where N_d is the number of detected defects, and N_{ts} is the number of total tested stories.

4.3.2 Calibrating model. Due to the missing of actual data, model calibration became a difficult step of modeling [17].

The main reason for the poor performance of the model is that the measurement of values for variables is unreasonable [3]. Therefore, the adjustment of the measurement method is extensively discussed for model calibration.

In our modeling, the calculation of individual's productivity for *implementation* phase is as Equation 2. The result of individual's productivity of implementation per day is shown in Table 4. When assigning the calculated value to the individual's productivity in the SD module, the simulated result is not good enough compared against the actual data (as shown in Figure 2).

Although the completion of a *story* is recorded on a certain day, part of the requirements is progressively completed daily from the initial reception of the requirements to the end. Therefore, when specifying the parameter value, modelers should consider whether the applied measurement methods can reflect the real process. For instance, individual's productivity of implementation is calculated as shown in Equation 6,

Table 4: Individual's productivity of implementation

Day 1	2	3	4	5	6	7	8
0	0	0.3333	0	0.0769	0.4286	0.080	0.240
Day 9	10	11	12	13	14	15	16
0.0417	0	0	1.2083	0.8947	1.0833	2	1

$$P'_{dev} = \frac{\sum_{i \subseteq T} N_{dsi}}{\sum_{i \subseteq T} N_{devi}} \tag{6}$$

where T represents an array containing each day from the start date till the current, N_{dsi} is the number of developed *stories* in the ith day and N_{devi} is the number of developers who implement these *stories* on the same day.

Table 5: Individual's productivity of implementation after adjustment

Day 1	2	3	4	5	6	7	8
0.2215	0.1904	0.1098	0.1098	0.0905	0.2216	0.1646	0.1888
Day 9	10	11	12	13	14	15	16
0.1578	0.1304	0.1792	0.3459	0.4119	0.4604	0.4945	0.5065

The reason for adjusting the measurement method is that requirements are implemented progressively every day throughout the *implementation* phase, instead of a once-off activity on a single

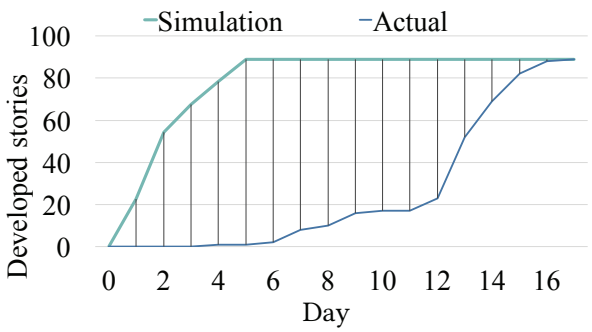


Figure 2: Comparison of simulation against actual data by single value range

day. The developed requirements are accordingly averaged to each day in the phase. Other productivity measures at the granularity of requirements are handled in the similar way. The adjusted result of individual's productivity is shown in Table 5. Figure3(a) shows a comparison between the simulated results after adjustment towards the actual process. Compared with Figure 2 that shows the simulation results against the actual process using a single range of values, it is evident that the adjusted measurement method outperforms single value range so that can achieve higher accuracy in measuring the productivity in simulation.

We selected two versions of the project, one was used for model calibration and the other was for model validation. The model's main variables and their values (presented in Table 6) were calculated from the raw data shown in Table 3. The *input* variables represent iteration specific information into the model, while the *calibration* variables specify one type of project information from the historical data.

The choice of the time span of historical data for calibration is a practical consideration for modelers. To improve the fidelity of the simulation model, we expected the time span of project life cycle could be long enough to model the actual behaviour rather than any temporal trend and yet capture the current reality [3], [28]. After consulting the PM and TPM of the selected project, we gained knowledge of the underlying changes. The modeled project was scheduled for monthly release and created at the beginning of 2019, whilst it experienced a migration of the code repository in April 2019. Since then, the project started to use TPMS to manage and visualize its test process. These changes lead to a limited period of time span of historical data being available to our modeling. More data on this project had to be accumulated until the end of 2019 for completing model calibration with the data from a set of versions.

4.4 Verification and validation

We employed two model Verification and Validation (V&V) methods identified in [14] for evaluating the process model.

Face validity. Based on the investigation of the enterprise's software process, the corresponding descriptive process model was created as the blueprint of the simulation model. Afterwards we applied face validity, which is defined as the degree to which test respondents view the content of a model as relevant to the software process being modeled [28], to verify the graphic descriptive process model by the modeling experts and the different roles of professionals involved in the development process.

Comparison against actual data. The simulated results of the hybrid process model are presented in Figure 3 in comparison with the actual data during *implementation* phase (shown in Figure 3(a)), *test* phase (Figure 3(b)), defect detection (Figure 3(c)), and regression testing (Figure 3(d)).

Both the modeling experts and the multiple roles of professionals who joined the V&V concluded that the hybrid model is able to closely reflect the actual process enactment.

4.5 Simulation based investigations

We used two metrics to evaluate the performance of the model [31]: Relative Error (RE), which measures the relative error between the simulation results and the actual data; and Mean Square Error (MSE), which measures the mean square error between them. They are defined as follows:

$$RE = \frac{|M_r - S_r(t_f)|}{M_r} \tag{7}$$

where M_r represents the actual data, t_f is constant when the phase is end, and S_f (t_f) represents the simulated result at t_f.

$$MSE = \frac{1}{N}\sum_{k=1}^{N}((A_r(t_k) + S_r(t_k))^2) \tag{8}$$

where N represents the number of data points, t_k represents the k-th day, A_r (t_k) is the actual data at t_k, and S_r (t_k) represents the simulation result at t_k. The performance of the hybrid model is shown in Table 7. For *implementation* phase, we used actual data of *stories* in the model, hence there is no relative error on the developed *stories* and the tested *stories*.

From the results shown in Table 7, we found that the performance of the hybrid model in predicting the time of the defect detected and the defect closed is better than the performance in predicting developed *stories* and tested *stories*. When analyzing the data, we observed that the duration from the detection to the closure of defect was getting shorter, which implies that the variables' values turn to be more stable. Therefore, the fidelity of the hybrid model for defect discovery and regression testing is higher than others.

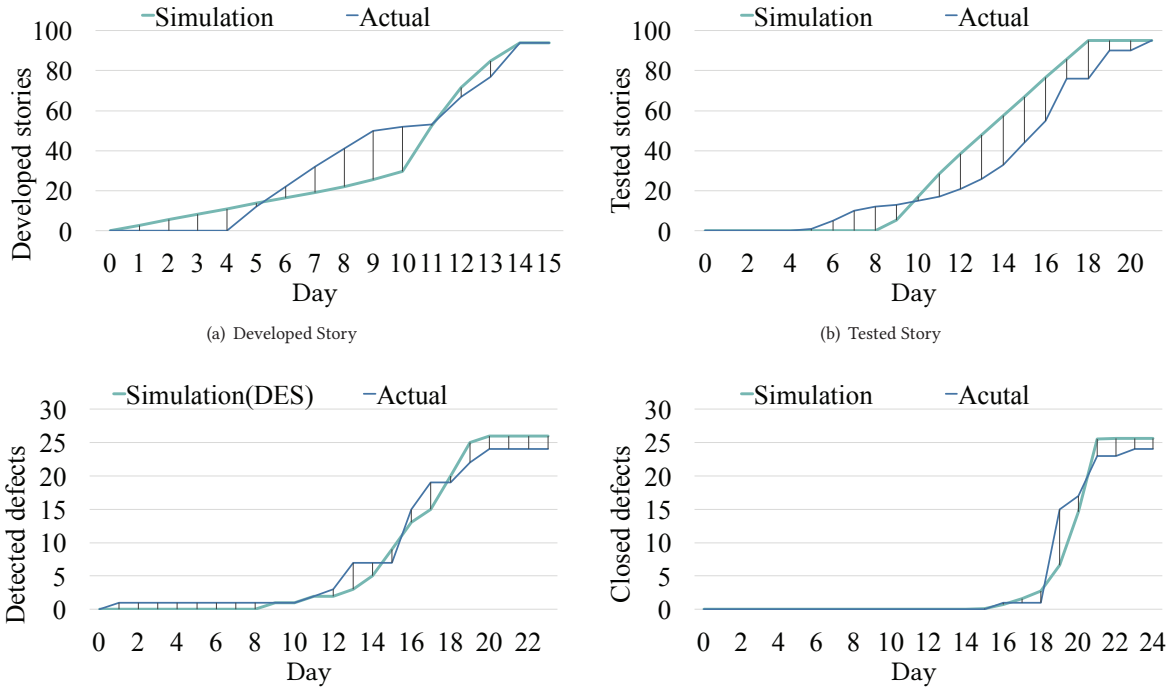
The hybrid process simulation model and the simulation engine have been deployed on the enterprise's online platform for project management. The model is able to predict the delivery cycle of a release with the input of variables including the number of initial *stories* and the team size to support PM in (re-)planning projects.

5 DISCUSSION

This section discusses our experiences in constructing the hybrid process simulation model. Considering the model calibration issues and real-world data problems reported in [3], [28], we discuss the difficulties we coped with in the industrial context in terms of understanding of data availability, investigation of data quality, and measurement of model variables.

Table 6: Major parameters & values of the hybrid process model

Variables	Variable type	Description	Unit	Project		
				Min	Max	Value
N_{t_s}	Input	Total number of planned stories per version.	Numeric	94	94	94
N_{t_dev}	Input	Total number of developers per version.	Numeric	33	33	33
P_{dev}	Calibration	Productivity of each developer.	Rate	0.1098	0.5065	random(0.1098,0.5065)
U_{dev}	Calibration	Utilization ratio of developers per day.	Percentage	0.5627	0.8281	random (0.5627,0.8281)
N_{test}	Input	Total number of testers per version.	Numeric	16	16	16
P_{test}	Calibration	Productivity of each tester.	Rate	0.1071	0.8387	random(0.1071,0.8387)
U_{test}	Calibration	Utilization ratio of tester per day.	Percentage	0.6251	0.923	random(0.6251,0.923)
N_{dd}	Calibration	Number of developers assigned defects per version.	Numeric	5	14	random(5,14)
T_{mdf}	Calibration	Time spent for fixing minor defect.	Days	2.176	3.257	random(2.176,3.257)
T_{sdf}	Calibration	Time spent for fixing suggestion defect.	Days	0.0432	20.88	random(0.0432,20.88)
N_{dr}	Calibration	Number of testers responsible for regression test per version.	Numeric	16	16	16
T_{mdt}	Calibration	Time spent for testing minor defect.	Days	2.176	3.257	random(2.176,3.257)
T_{sdt}	Calibration	Time spent for testing suggestion defect.	Days	0.0432	20.88	random(0.0432,20.88)



(a) Developed Story

(b) Tested Story

(c) Detected Defect

(d) Closed Defect

Figure 3: Comparison between the simulation results and the actual data

5.1 Reciprocal understanding on data availability

Investigating and determining what data is available for process modeling and simulation is an iterative process that may go back and forth between the analysis of the collected data and the communications with the project team members. In the initial stage, we were not aware of what data from the enterprise could be available to us, whilst the practitioners involved in the collaboration had no idea what data is needed for modeling. In order to precisely model

Table 7: Evaluation result by relative error and mean squared error

	Developed Story	Tested Story	Detected Defect	Closed Defect
RE	0	0	0.0769	0.0769
MSE	1963.4162	3318.5247	67	95.8219

and simulate the different activities in the development process, we interviewed the selected staff who play different roles in the project to learn the project/process management systems they use in their daily work and the data accessible from these systems. We tried to analyze the data from the two identified repositories (RMS and TPMS) at the beginning, but quickly found there was no standard and detailed description for each data field. Hence, we began a series of lengthy follow-up interviews to understand or confirm the data structure and its relationship to the development process. Once a complete understanding of the available data was reached, we perceived that the data from these repositories is not sufficient for modeling. For some abnormal data in particular, we had to learn about their behavior patterns behind the data through interviews. During the process of interviewing staff from different teams, some new data sources emerged for the references of the development process, e.g., some offline documents, which the developers maintained in real time. As a result, we discarded some data from online repositories due to the quality concerns, and appended extra data from the newly identified data sources. From our modeling experience, the understanding of the data for process modeling would be a spiraling and iterative process in industry. This process may require the close cooperation with multiple teams, which is lengthy but inevitable. As another lesson learned, early communications with enough teams will gain insights into the work patterns of different roles, which helps to reduce unnecessary effort and rework in the follow-up phases of the modeling. Process modeling in reality could be difficult for software companies to arrange many a staff to be interviewed in a short period of time, and for modelers to thoroughly consider enough problems and unexpected issues at the beginning. For the practical situations of process modeling, frequent iterations in small feedback loops may work for quick understanding.

5.2 Investigation of data quality

As reported in other research [28], data quality for process management is far from desire in companies, in particular for the manually recorded data, such as updates of the status of stories and defects in our study. In order to correctly understand the data and further identify the errors in the data, we interviewed the developers and the testers from three different departments in the enterprise. We found the processes recovered by their recorded data in online repositories are inconsistent with the processes executed. Fortunately, we could confirm the actual process to be modeled with one team who kept detailed logs using offline documentation. The reason for this is that although the use of online systems is mandatory in the enterprise, keeping logs has become the long-standing behavior patterns of this team. From our experience, the evaluation of data quality relies on not only the data analysis, but perhaps more importantly, the understanding of the behaviors of the actual data recorders.

5.3 Calibration of model variables

Model calibration is another practical difficulty [3], [28]. The variables to be calibrated may change their values dynamically, it is hard to ensure these variables in real projects could fit in a given distribution [15], [5]. In this study, we collected data from a set of released versions, and extracted the raw data from different data sources as presented in Table 3 for calibrating model variables. Taking *development rate* as an example, it is calculated with *number of developers*, *individual productivity*, and *developer utilization*, among which the latter two may vary continuously during the development. We employed Monte Carlo Simulations [28], [20], [33], [32] to imitate the fluctuation range of the variables by simulation within a certain confidence interval.

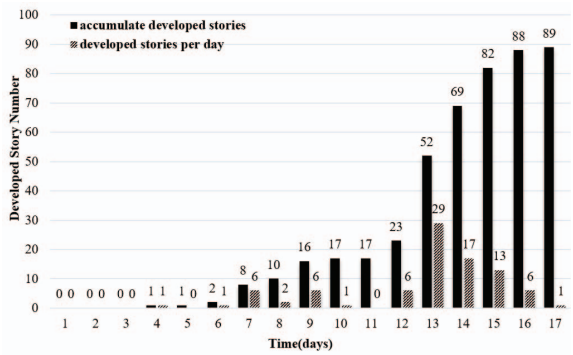


Figure 4: Distribution of the developed stories

In our simulation model, the simulation pace was configured as one day. Hence, the *individual productivity* is measured by number of stories implemented per person day. However, it would lead to two problems as shown in Figure 4. First, it is common in real processes that no requirements are implemented in the initial stages of development. Next, the rate of requirement completion would increase dramatically near the end of the development. To improve the accuracy of the simulation, we used the moving average method [7] to adjust the productivity as defined in Equation 6.

6 CONCLUSIONS

Over the past two decades, Software Process Simulation Modeling (SPSM) has demonstrate its potential to be a powerful tool to support quantitative process management. In particular with the popularity of DevOps in recent years, a large amount of data generated by automated development tools during the software process is available for researchers and practitioners to quantitatively analyze and model software processes. Considering that process simulation can facilitate project tracking and (re-)planning of the software process, it is applied to predict the delivery of requirements in industry cases in our collaboration with a global software enterprise. However, it is challenging for process modelers to collect desired

process data from the established software repositories and software artifacts that are primarily designed for artifact management rather than process management and modeling. The complexity of hybrid simulation and the lack of modeling experience in industry inspire us to report on our modeling practice, challenges, and experiences. Based on the collaboration with the software enterprise, we elaborate the steps in action for constructing the hybrid process simulation model. In particular, we discuss our solutions and their impact on modeling within the context of the concrete industrial process scenarios and available data. As many software projects still suffers from high effort as well as low accuracy in quantitative process management, our experience on hybrid process simulation modeling may offer a useful reference for modelers and organizations who intent to apply this approach to leverage their process management in industrial settings. As the next phase of our work, we plan to apply other alternative solutions, e.g., requirements traceability recovery, to further improve the quality of process data and the accuracy of simulation model. We encourage future research to investigate on assisting software organizations to adopt (hybrid) process simulation in a cost-effective manner.

ACKNOWLEDGMENT

This work is supported by the National Natural Science Foundation of China (Grant No.61572251).

REFERENCES

[1] Tarek Abdel-Hamid and Stuart E. Madnick. 1991. *Software Project Dynamics: An Integrated Approach.* Prentice-Hall, Inc.

[2] Rizwan Ahmed, Tracy Hall, Paul Wernick, Stewart Robinson, and M Shah. 2008. Software process simulation modelling: A survey of practice. *Journal of simulation* 2, 2 (2008), 91–102.

[3] Nauman Bin Ali and Kai Petersen. 2012. A Consolidated Process for Software Process Simulation: State of the Art and Industry Experience. In *Proceedings of the 2012 EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA'12).* IEEE, Cesme, Izmir, Turkey, 327–336.

[4] Xu Bai, Liguo Huang, He Zhang, and Supannika Koolmanojwong. 2012. Hybrid modeling and simulation for trustworthy software process management: a stakeholder-oriented approach. *Journal of Software: Evolution and Process* 24, 7 (2012), 721–740.

[5] Tobias Baum, Fabian Kortum, Kurt Schneider, Arthur Brack, and Jens Schauder. 2016. Comparing Pre Commit Reviews and Post Commit Reviews Using Process Simulation. In *Proceedings of the 2016 International Conference on Software and Systems Process (ICSSP '16).* ACM, Austin, Texas, 26–35.

[6] Sally C. Brailsford, Shivam M. Desai, and Joe Viana. 2010. Towards the holy grail: Combining system dynamics and discrete-event simulation in healthcare. In *Proceedings of the 2010 Winter Simulation Conference (WSC'10).* IEEE, Baltimore, Maryland, US, 2293–2303.

[7] Kurt Brännäs and Andreia Hall. 2001. Estimation in integer-valued moving average models. *Applied Stochastic Models in Business and Industry* 17, 3 (2001), 277–291.

[8] KeungSik Choi, Doo-Hwan Bae, and TagGon Kim. 2006. An approach to a hybrid software process simulation using the DEVS formalism. *Software Process: Improvement and Practice* 11, 4 (2006), 373–383.

[9] Paolo Donzelli and Giuseppe Iazeolla. 2001. Hybrid simulation modelling of the software process. *Journal of Systems and Software* 59, 3 (2001), 227 – 235.

[10] Tillal Eldabi, Mariusz Balaban, Sally C. Brailsford, Navonil Mustafee, Richard E. Nance, Bhakti Stephan Onggo, and Robert G. Sargent. 2016. Hybrid Simulation: Historical lessons, present challenges and futures. In *Proceedings of the 2016 Winter Simulation Conference (WSC'16).* IEEE, Washington, DC, USA, 1388–1403.

[11] David Arthur Fahrland. 1970. Combined discrete event continuous systems simulation. *SIMULATION* 14, 2 (1970), 61–72.

[12] Reinaldo Cabral Silva Filho and Ana Regina Cavalcanti da Rocha. 2010. Towards an Approach to Support Software Process Simulation in Small and Medium Enterprises. In *Proceedings of the 2010 EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA'10).* IEEE, Lille, France, 297–305.

[13] Chao Gao, He Zhang, and Shu Jiang. 2015. Constructing Hybrid Software Process Simulation Models. In *Proceedings of the 2015 International Conference on Software and System Process (ICSSP'15).* ACM, New York, NY, USA, 157–166.

[14] Haojie Gong, He Zhang, Dexian Yu, and Bohan Liu. 2017. A Systematic Map on Verifying and Validating Software Process Simulation Models. In *Proceedings of the 2017 International Conference on Software and System Process (ICSSP'17).* ACM, Paris, France, 50–59.

[15] Daniel Xavier Houston. 2000. *A Software Project Simulation Model for Risk Management.* Ph.D. Dissertation. Advisor(s) Mackulak, G.

[16] Bojan Jovanoski, Robert Minovski, Siegfried Voessner, and Gerald Lichtenegger. 2012. Combining system dynamics and discrete event simulations - Overview of hybrid simulation models. *Journal of Applied Engineering Science* 10 (2012), 135–142.

[17] Marc I. Kellner, Raymond J. Madachy, and David M. Raffo. 1999. Software process simulation modeling: Why? What? How? *Journal of Systems and Software* 46, 2 (1999), 91 – 105.

[18] Diana Kirk and Stephen MacDonell. 2009. A Simulation Framework to Support Software Project (Re)Planning. In *Proceedings of the 2009 EUROMICRO Conference on Software Engineering and Advanced Applications (SEAA'09).* IEEE, Patras, Greece, 285–292.

[19] Peter B Lakey. 2003. A hybrid software process simulation model for project management. In *Proceedings of the 2003 International Workshop on Software Process Simulation Modeling (ProSim'03).* Portland, OR, USA.

[20] Raymond J. Madachy. 2007. *Software process dynamics.* John Wiley & Sons.

[21] Robert H. Martin and David M. Raffo. 2000. A model of the software development process using both continuous and discrete models. *Software Process: Improvement and Practice* 5, 2-3 (2000), 147–157.

[22] Robert H. Martin and David M. Raffo. 2001. Application of a hybrid process simulation model to a software development project. *Journal of Systems and Software* 59, 3 (2001), 237–246.

[23] Javier Navascués, Isabel Ramos, and Miguel Toro. 2009. A Hybrid Model for Dynamic Simulation of Custom Software Projects in a Multiproject Environment. In *Proceedings of the International Conference on Software Process: Trustworthy Software Development Processes.* Springer, Vancouver, Canada, 173–185.

[24] Holger Neu and Ioana Rus. 2003. Reuse in software process simulation modeling. In *Proceedings of the 2003 International Workshop on Software Process Simulation Modeling (ProSim'03).* Portland, OR, USA, 1–4.

[25] Frank Padberg and David Weiss. 2011. Model-Based Scheduling Analysis for Software Projects. In *Proceedings of the 2011 IEEE 35th Annual Computer Software and Applications Conference Workshops (COMPSACW'11).* IEEE, Munich, Germany, 304–309.

[26] Luis Rabelo, Magdy Helal, Albert Jones, and Hyeung-Sik Min. 2005. Enterprise simulation: a hybrid system approach. *International Journal of Computer Integrated Manufacturing* 18 (2005), 498–508.

[27] David M. Raffo, Robert Ferguson, Siri-on Setamanit, and Bhuricha Deen Sethanandha. 2007. Evaluating the Impact of the QuARS Requirements Analysis Tool Using Simulation. In *Proceedings of the 2007 International Conference on Software Process (ICSP'07).* Springer, 307–319.

[28] David M. Raffo and Marc I. Kellner. 2000. Empirical analysis in software process simulation modeling. *Journal of Systems and Software* 53, 1 (2000), 31 – 41.

[29] Ioana Rus, Holger Neu, and Jürgen Münch. 2003. A Systematic Methodology for Developing Discrete Event Simulation Models of Software Development Processes. In *Proceedings of the 2003 International Workshop on Software Process Simulation Modeling (ProSim'03).* Portland, OR, USA.

[30] Robert E. Shannon. 1998. Introduction to the art and science of simulation. In *Proceedings of the 1998 Winter Simulation Conference (WSC'98)*, Vol. 1. IEEE, Washington, DC, USA, 7–14.

[31] Sy-Yen Kuo, Chin-Yu Huang, and M. R. Lyu. 2001. Framework for modeling software reliability, using various testing-efforts and fault-detection rates. *IEEE Transactions on Reliability* 50, 3 (2001), 310–320.

[32] Richard Turner, Raymond J. Madachy, Dan Ingold, and Jo Ann Lane. 2012. Modeling kanban processes in systems engineering. In *Proceedings of the 2012 International Conference on Software and System Process (ICSSP'12).* IEEE, Zurich, Switzerland, 23–27.

[33] Masood Uzzafer. 2013. A contingency estimation model for software projects. *International Journal of Project Management* 31, 7 (2013), 981 – 993.

[34] Jayendran Venkateswaran, Young-Jun Son, Albert T. Jones, and Hyeung-Sik Min. 2006. A hybrid simulation approach to planning in a VMI supply chain. *International Journal of Simulation and Process Modelling* 2, 3/4 (2006), 133–149.

[35] He Zhang, Barbara A. Kitchenham, and Dietmar Pfahl. 2008. Reflections on 10 Years of Software Process Simulation Modeling: A Systematic Review. In *Making Globally Distributed Software Development a Success Story.* Springer, 345–356.

[36] He Zhang, Barbara A. Kitchenham, and Dietmar Pfahl. 2010. Software Process Simulation Modeling: An Extended Systematic Review. In *Proceedings of the 2010 International Conference on New Modeling Concepts for Today's Software Processes: Software Process.* Springer, Paderborn, Germany, 309–320.

[37] Jafri Zulkepli and Tillal Eldabi. 2015. Towards a framework for conceptual model hybridization in healthcare. In *Proceedings of the 2015 Winter Simulation Conference (WSC'15).* IEEE, Huntington Beach, CA, USA, 1597–1608.